1. Analysis

i. Structure of an ANN

The input, hidden, and output layers are important structural pieces of the ANN. The input layer is concerned with receiving raw data from the domain (Baheti, 2021), such as monthly fees, contract type, tenure, internet usage, and customer service calls. For Telco, each input neuron corresponds to one of the customer-related attributes. This is the data before any computations are executed on it.

The hidden layers provide a level of abstraction and perform the bulk of the computational work. A neural network can have one or multiple hidden layers, and each layer consists of neurons that make the inputs something the output layer can use (Ghorakavi, 2026). Inputs are transformed into new representations that allow the network to find complex patterns within the data.

The output layer, as the name suggests, outputs the model's prediction (Ghorakavi, 2026). Regarding customer churn, a binary classification problem, the output layer usually uses a single neuron to provide a binary probability value of 0 or 1.

Weights and biases play an important role in how these layers function. Weights control the strength and influence of each input feature on the final decision (Lee, 2021). An example of an input that might be weighted higher than others is internet usage. If a customer is not using the internet that much, they are more likely to discontinue service. But other factors might have more influence over the other, regardless of whether another factor is more or less present. In this case, biases add values to the weighted sum that shift the decision-making threshold. Biases allow neurons to activate even with weaker inputs (Lee, 2021).

The activation function decides whether or not a function should be activated (Baheti, 2021). It adds nonlinearity to the network. Hidden layers need activation functions because nonlinear relationships would be unlearnable without them (Baheti, 2021). ReLU is a favored function for hidden layers because it is efficient. Furthermore, sigmoid functions are also common for the output layer in binary classification scenarios, like this one. Sigmoid functions will be featured in the code.

ii. How Forward Propagation and Backpropagation Work in Learning

Note that the learning process of an ANN follows a three-stage cycle: input computation, output generation, and iterative refinement (Ghorakavi, 2026). This is important to know for propagation. Forward propagation is the flow of information from the input layer toward the output layer. Each neuron calculates a weighted sum of its inputs, adds bias, and then applies an activation function to generate an output signal for the next layer (Lee, 2021). This continues until a prediction is generated by the output layer.

Backpropagation takes the prediction and uses a loss function to calculate the error between the predicted value and the real outcome (Ghorakavi, 2026). The error then takes a backwards pass into the network, where it determines how much weight and bias at each neuron contributed to the mistake in judgment (Lee, 2021). This is the mechanism that improves a model over time. The network calculates gradients and applies an optimization algorithm to update the weights and biases to better mitigate the measure of error in predictions, also known as loss (Lee, 2021). Multiple iterations, also known as epochs, allow the model to hone in on a set of parameters that optimize the prediction accuracy.

iii. Logic Gate Implementation in Python

Basic logic gates, such as AND, OR, and NOT, can be implemented using a single neuron. This is because they are considered linearly separable, which means that a single straight line can separate inputs that result in either a 0 or a 1 as the output (Starmer, 2020). The exception to this is the XOR gate (exclusive OR), where the outputs are only 1 when the inputs are different (e.g., (0, 1), (1, 0)). To remedy this, XOR gates require at least one hidden layer and a nonlinear activation function (Baheti, 2021). The hidden layer allows the network to project data into a higher dimension where a linear boundary can then be drawn. This is why XOR requires a more complex architecture.

Logic gate source code:

```
import numpy as np

# A single neuron function
# inputs are the binary values of 0 or 1
# weights assign importance to each input
# bias is the threshold to trigger the neuron
def perceptron(inputs, weights, bias):
    # Step 1: Calculates the weighted sum (Z = w1*x1 + w2*x2 + ... + b)
    z = np.dot(inputs, weights) + bias
    # Step 2: Applies a step activation function (output 1 if Z >= 0. else
0).
    return 1 if z >= 0 else 0

# 1. Implementation of AND, OR, and NOT gates
print("Linear Logic Gates")
# Manually defined weights and biases that satisfy the logic gate truths
gates = {
    "AND": {"w": [1, 1], "b": -1.5}, # Requires both inputs to be 1 to
overcome -1.5.
    "OR":  {"w": [1, 1], "b": -0.5}, # Only requires one input to be 1 to
overcome -0.5.
    "NOT": {"w": [-1],   "b": 0.5}   # Flips 0 to positive, flips 1 to
negative.
```

```
}

for gate, p in gates.items(): #For loop
    print(f"\n{gate} Gate Results:")
    # Defines inputs based on whether it is a 1-input (NOT) or 2-input
gate
    tests = [[0, 0], [0, 1], [1, 0], [1, 1]] if gate != "NOT" else [[0],
[1]]
    for x in tests:
        output = perceptron(np.array(x), np.array(p['w']), p['b'])
        print(f"Input {x} -> Output {output}")

# 2. XOR implementation that uses a hidden layer architecture
def xor_logic(x1, x2):
    # XOR is solved by combining OR and NAND logic into an AND gate
    # Neuron 1 (hidden) that acts as an OR gate
    h1 = perceptron([x1, x2], [1, 1], -0.5)
    # Neuron 2 (hidden) that acts as a NAND gate
    h2 = perceptron([x1, x2], [-1, -1], 1.5)
    # Neuron 3 (output) that performs AND logic on the two hidden neurons
    # This architecture creates the non-linear boundary needed for XOR
    return perceptron([h1, h2], [1, 1], -1.5)

print("\nXOR Gate")
for x in [[0, 0], [0, 1], [1, 0], [1, 1]]:
    print(f"Input {x} -> Output {xor_logic(x[0], x[1])}")
```

Logic gate output:

```
Linear Logic Gates

AND Gate Results:
Input [0, 0] -> Output 0
Input [0, 1] -> Output 0
Input [1, 0] -> Output 0
Input [1, 1] -> Output 1

OR Gate Results:
Input [0, 0] -> Output 0
Input [0, 1] -> Output 1
Input [1, 0] -> Output 1
Input [1, 1] -> Output 1

NOT Gate Results:
Input [0] -> Output 1
Input [1] -> Output 0

XOR Gate
Input [0, 0] -> Output 0
Input [0, 1] -> Output 1
Input [1, 0] -> Output 1
Input [1, 1] -> Output 0
```

2. Neural Network Implementation

i. Data Preprocessing for the Dataset

Before full implementation, data preprocessing is an important step. First, categorical values should be converted to numerical ones. Inputs, such as tenure or contract type, must be represented numerically. Next, differing numerical scales, such as tenure in months and internet usage in minutes, should be differentiated. Failure to do so can lead to unusually poor predictions. Another important step is to differentiate between the training and testing sets. The model should learn from training data, while testing sets should be used to evaluate the model's performance.

ii. Neural Network Model in Python

Customer churn model source code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import confusion_matrix, classification_report

# i. Data preprocessing
# Link to the dataset
url =
"https://huggingface.co/Ray0071/Hackathon/raw/main/WA_Fn-UseC_-Telco-Custo
mer-Churn.csv"
df = pd.read_csv(url)

# Converts TotalCharges to numeric
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df.dropna(inplace=True) # Remove rows with missing data
df.drop('customerID', axis=1, inplace=True) # ID is unnecessary for
training

# Handles categorical variables using label encoding
le = LabelEncoder()
for col in df.select_dtypes(include=['object']).columns:
    df[col] = le.fit_transform(df[col])

# Separates target (the churn) from features
X = df.drop('Churn', axis=1).values
y = df['Churn'].values.reshape(-1, 1)

# Feature scaling
# Neural networks converge faster when data is normalized (mean=0, std=1)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Splits into 80% training and 20% testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# ii. Manual neural network using NumPy
```

```
class ManualNeuralNetwork:
    def __init__(self, input_dim, hidden_dim, output_dim):
        # Initializes the weights with small, random values to prevent
symmetry
        self.W1 = np.random.randn(input_dim, hidden_dim) * 0.01
        self.b1 = np.zeros((1, hidden_dim))
        self.W2 = np.random.randn(hidden_dim, output_dim) * 0.01
        self.b2 = np.zeros((1, output_dim))

    def sigmoid(self, z):
        # Activation function to squash values between 0 and 1
        return 1 / (1 + np.exp(-z))

    def sigmoid_derivative(self, a):
        # Used during backpropagation to calculate gradients
        return a * (1 - a)

    def train(self, X, y, epochs, learning_rate):
        for epoch in range(epochs):
            # Phase 1: forward propagation
            # Layer 1 (hidden): Weighted sum + activation
            Z1 = np.dot(X, self.W1) + self.b1
            A1 = self.sigmoid(Z1)
            # Layer 2 (output): Weighted sum + activation
            Z2 = np.dot(A1, self.W2) + self.b2
            A2 = self.sigmoid(Z2)

            # Phase 2: Backpropagation (error calculation)
            # Calculates the output error (predicted - actual)
            error_out = A2 - y

            # Calculates gradients for Weight 2 and Bias 2 (Output Layer)
            # Derivative of loss regarding W2
            dW2 = np.dot(A1.T, error_out)
            db2 = np.sum(error_out, axis=0, keepdims=True)

            # Calculates error for the hidden layer
            error_hidden = np.dot(error_out, self.W2.T) *
self.sigmoid_derivative(A1)
```

```
            # Calculates gradients for Weight 1 and Bias 1 (Hidden Layer)
            dW1 = np.dot(X.T, error_hidden)
            db1 = np.sum(error_hidden, axis=0, keepdims=True)

            # Phase 3: Updates the weights
            # Adjusts the parameters in the opposite direction of the
gradient
            self.W2 -= learning_rate * dW2
            self.b2 -= learning_rate * db2
            self.W1 -= learning_rate * dW1
            self.b1 -= learning_rate * db1

            # Prints progress every 200 epochs
            if epoch % 200 == 0:
                loss = np.mean(np.square(error_out)) # Mean squared error
for tracking
                print(f"Epoch {epoch} Loss: {loss:.4f}")

    def predict(self, X):
        # Runs a single forward pass to generate final predictions.
        A1 = self.sigmoid(np.dot(X, self.W1) + self.b1)
        A2 = self.sigmoid(np.dot(A1, self.W2) + self.b2)
        # Converts probabilities to binary 0 or 1
        return (A2 > 0.5).astype(int)

# Initializes and trains the network
# input_dim = number of features. hidden_dim = 12 neurons. output_dim = 1
(churn or not)
nn = ManualNeuralNetwork(input_dim=X_train.shape[1], hidden_dim=12,
output_dim=1)
nn.train(X_train, y_train, epochs=1000, learning_rate=0.001)

# iii. Model evaluation
y_pred = nn.predict(X_test)
print("\nManual ANN Evaluation Results")
print(f"Accuracy Score: {np.mean(y_pred == y_test):.4f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nPerformance Report:")
print(classification_report(y_test, y_pred))
```

Customer churn model output:

```
Epoch 0 Loss: 0.2538
Epoch 200 Loss: 0.1347
Epoch 400 Loss: 0.1355
Epoch 600 Loss: 0.1330
Epoch 800 Loss: 0.1311

Manual ANN Evaluation Results
Accuracy Score: 0.7818

Confusion Matrix:
[[879 154]
 [153 221]]

Performance Report:
              precision    recall  f1-score   support

           0       0.85      0.85      0.85      1033
           1       0.59      0.59      0.59       374

    accuracy                           0.78      1407
   macro avg       0.72      0.72      0.72      1407
weighted avg       0.78      0.78      0.78      1407
```

iii. Model Evaluation

This implementation shows a neural network applied to Telco's situation. The accuracy is measured as a percentage of correct predictions out of all the customers in the test set. The confusion matrix provides an intuitive breakdown of true positives, true negatives, false positives, and false negatives. As for performance results, the loss and accuracy results can be used to improve the model and avoid overfitting. Ideally, overfitting should be avoided.

Word count: 815

References

Baheti, P. (2021, May 27). *Activation functions in neural networks [12 types & use cases]*. V7. https://www.v7darwin.com/blog/neural-networks-activation-functions

Ghorakavi, V. (2026, July 4). *Introduction to neural networks*. GeeksforGeeks. https://www.geeksforgeeks.org/deep-learning/neural-networks-a-beginners-guide/

Lee, F.F. (2021, October 6). *What is a neural network?* IBM. https://www.ibm.com/think/topics/neural-networks

Starmer, J. [StatQuest with Josh Starmer]. (2020, August 31). *Neural networks pt. 1: Inside the black box* [Video]. YouTube. https://www.youtube.com/watch?v=CqOfi41LfDw